

EE332 Digital System Design

Dr. Yi Zeng
Assistant Professor,
Department of Electrical and Electronic Engineering
Office: S236, South Tower of COE
Email: zengyi@sustech.edu.cn

Text Book

- P.P. Chu, RTL hardware design using VHDL, John Wiley & Sons, 2006.

Reference Book

- S. Yalamanchili, VHDL: A Starter's Guide, Prentice Hall, 2005
- Charles H. Roth, Jr. and Lizzy Kurian John: Digital Systems Design with VHDL, Publishing house of electronics Industry

Lecture Plan

- Lectures – 32 hours
- Lab exercises – 18 hours
- Two-person group projects – 14 hours. Sample group projects:
 - remote control electronic piano
 - smart phone control electronic motor
 - intelligent chess robot
 - intelligent unmanned grounded vehicle
 - wearable sports device
 - etc.

Pre-requisites:

- Digital Electronics

Examinations:

- Final Exam: 40% (open book, 2 hours)
- Continuous Assessment: 60%
 - Lab exercises and report: 30%
 - Group project and report: 30%
- No plagiarism is tolerated. Zero marks are given to the part involving plagiarizing.**

Outline

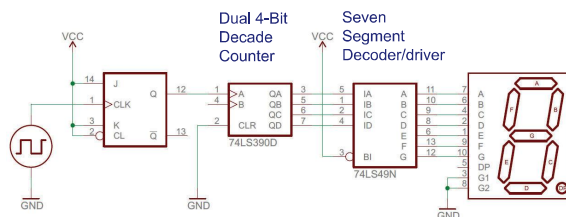
- Introduction
- Data Objects and Operands
- Concurrent statement
- Sequential statement
- Modeling Structure
- Modeling at the RT level
- Modeling at the FSMD level
- Parameterized Design
- Pipeline Design
- Subprograms, package and library
- Synthesis Of VHDL Code
- Clock and Synchronization

Chapter 1 Introduction

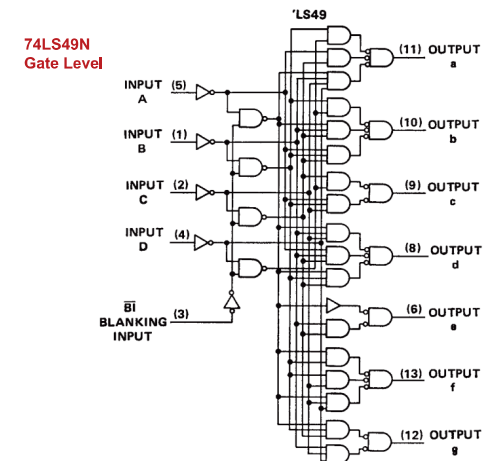
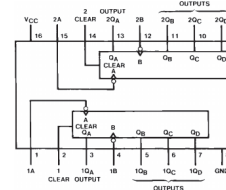
- Digital system design
 - System representation
 - Level of abstraction
 - Device technologies
- What is VHDL
- Basic VHDL concept via an example

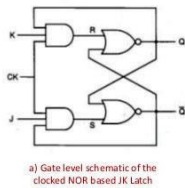
1.1 Introduction to digital system design

- Examples of Digital System: system level, or more accurately, register transfer level (RTL).

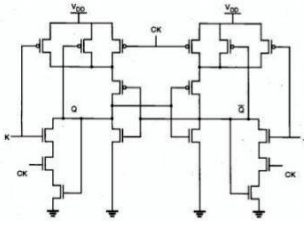


74LS390D
Dual 4-Bit Decade Counter



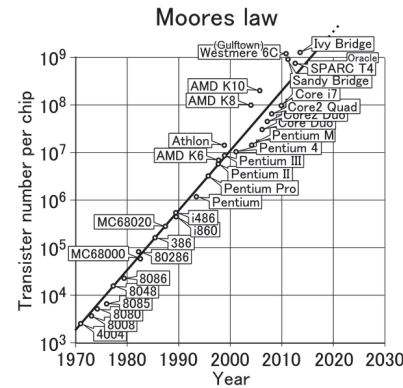


a) Gate level schematic of the clocked NOR based JK Latch



b) CMOS realization of the JK Latch

10



11



12

Abstraction

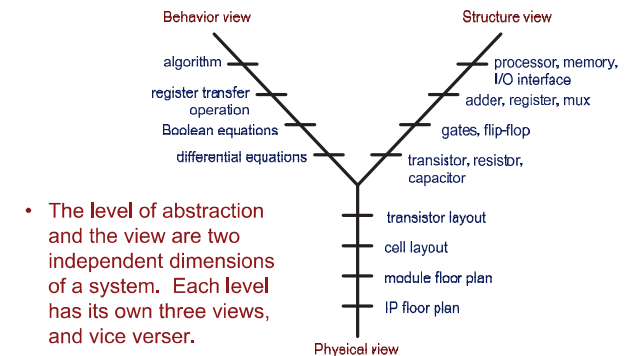
- A key method of managing complexity is to describe a system in several levels of abstraction.
- An **abstraction** is a simplified model of the system, showing only the selected features and ignoring the associated details.
 - Transistor level
 - Gate level
 - Register transfer (RT) level
 - Processor level

13

System Representation (View)

- View:** different perspectives of a system.
 - Behavior view
 - describe the functionalities and i/o behavior
 - treat the system as a black box
 - Structural view
 - describe the internal implementation (components and interconnections)
 - essentially block diagram
 - Physical view
 - add more info to structural view: component size, component location, routing wires
 - e.g. layout of a printed circuit board

14



- The level of abstraction and the view are two independent dimensions of a system. Each level has its own three views, and vice versa.

15

Characteristics of each abstraction level

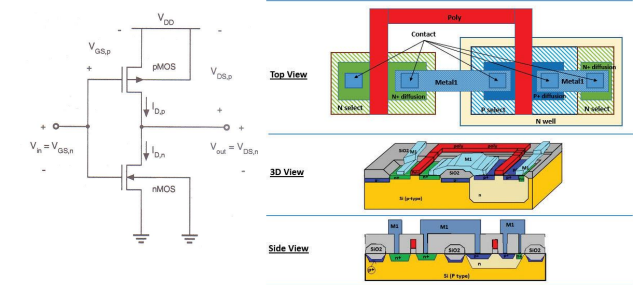
	Typical blocks	Signal representation	Time representation	Behavioral description	Physical description
Transistor	transistor, resistor	voltage	continuous function	differential equation	transistor layout
Gate	and, or, xor,	logic 0 or 1	propagation delay	Boolean equation	cell layout
RT	adder, mux, register	integer, system state	clock tick	generalized FSM	RT-level floor plan
Processor	processor, memory	abstract data type	event sequence	natural language	IP-level floor plan

16

Device Technology

- Fabrication of an IC**
 - Transistors and connection are made from many layers (typical 10 to 15 in CMOS) built on top of one another
 - Each layer has a special pattern defined by a mask
 - An important aspect of IC is the length of a smallest transistor that can be fabricated
 - It is measured in micron (μm , 10^{-6} meter) in early time, but now in nano (nm , 10^{-9} meter), and known as the **minimum feature size**.
 - e.g. we may say an IC is built with $0.6 \mu\text{m}$ process
 - The process continues to improve, as witnessed by Moore's law
 - The state-of-art process approaches a single digit of nm, for example 3nm.

17

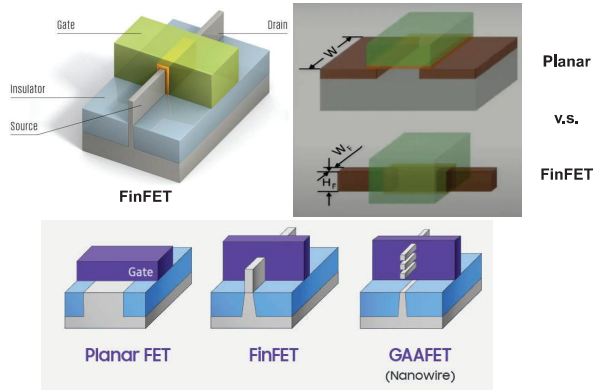


18

Device Technology Development Trend

- **More Moore**
 - Shrinking in the size (the channel length) of transistors
 - For example, TSMC (Taiwan Semiconductor Manufacturing Company) owns all the product lines appeared since 1980's, including 0.5μm, 0.35μm, 0.25μm, 0.18μm, 0.13μm , 90nm, 65nm, 40nm, 28nm, 22nm, 20nm, 16nm, 10nm, 7nm, 5nm, 3nm.
- **More Than Moore**
 - Structure innovation, for example 3D integrate, including FinFET, GAA (Gate-All-Around) FET, etc.

19

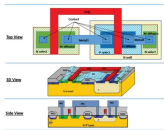


20

- **Classification of Device technologies:**
 - Where customization is done:
 - in a fab (fabrication facility): ASIC (Application Specific IC)
 - in the “field”: Non-ASIC
 - Classification
 - Full custom ASIC
 - Standard-cell ASIC
 - Gate array ASIC
 - Reprogrammable logic device: FPGA (field programmable gate array)

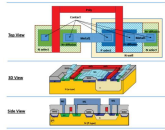
21

- **Full-custom ASIC**
 - All aspects (e.g. size of a transistor) of a circuit are tailored for a particular application.
 - Circuit fully optimized
 - Design extremely complex and involved.
 - Only feasible for small components
 - Masks needed for all layers



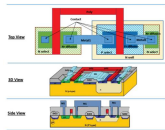
22

- **Standard-cell ASIC**
 - Circuit made of a set of pre-defined logic, known as standard cells.
 - E.g. basic logic gates, 1-bit adder, DFF etc.
 - Layout of a cell is pre-determined, but layout of the complete circuit is customized
 - Masks needed for all layers.



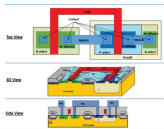
23

- **Gate array ASIC**
 - Circuit is built from an array of a single type of cell (known as base cell)
 - Base cell are pre-arranged and placed in fixed positions, aligned as one- or two- dimensional array
 - More sophisticated components (macro cells) can be constructed from base cells
 - Masks needed only for metal layers (connective wires)



24

- **Field programmable gate array**
 - Device consists of an array of generic logic cells and general interconnect structure
 - Logic cells and interconnect can be “programmed” by utilizing semiconductor fuses or “switches”
 - Customization is done in the “field”
 - No custom mask needed



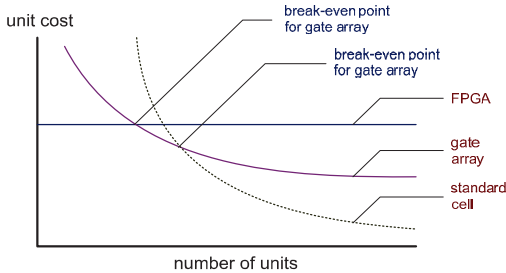
25

Comparison of device technologies

	Custom (Full Custom)	Cell-based (Standard Cell)	Pre-diffused (Gate Array)	Prewired (CPLD, FPGA)
Density	Very High	High	High	Medium - Low
Performance	Very High	High	High	Medium - Low
Flexibility	Very High	High	Medium	Low
Design Time	Very Long	Short	Short	Very Short
Manufacturing time	Medium	Medium	Short	Very Short
Cost – Low /High Volume	Very High/Low	High/Low	High/Low	Low/High

26

Cost: $C_{unit} = C_{per_part} + C_{one_time} / units_produced$

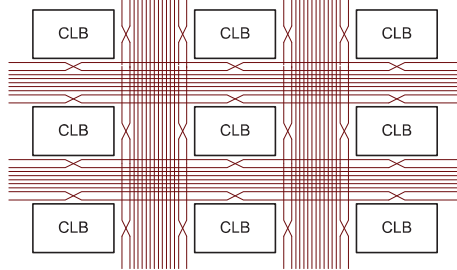


Comparison of per unit cost

27

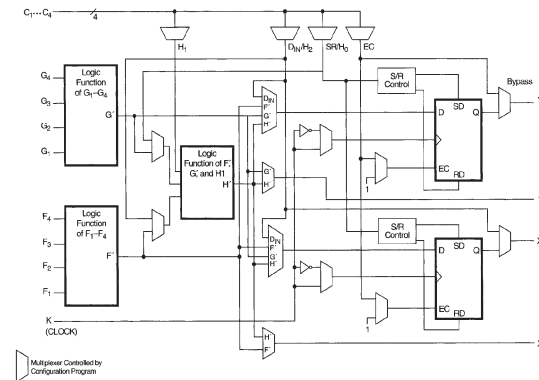
Field programmable gate array (FPGA)

- Ideally, the interconnection between the millions of transistors can be reconfigured to realize different applications.

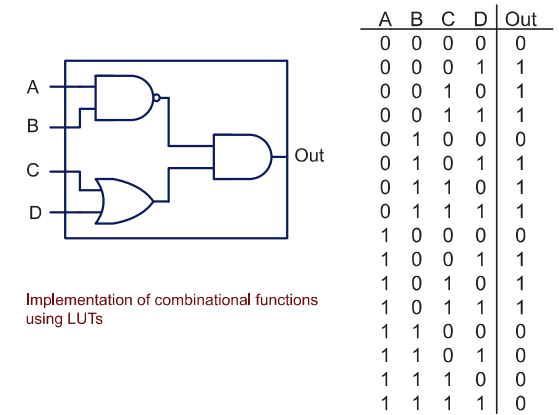


28

CLB - Configurable logic block



29



Implementation of combinational functions using LUTs

30

1.2 What is VHDL?

- VHSIC Hardware Description Language (VHDL)**
 - Very High-Speed Integrated Circuit program (VHSIC)
 - A computer language for **documenting** and **simulating** circuits, and describing circuits for **synthesis**.
 - A high level programming language with specialized constructs for **modeling hardware**.

31

History of VHDL

- Intermetrics, TI and IBM under US DoD contract 1983-1985: VHDL 7.2
- IEEE standardization: VHDL 1076-1987
- First synthesized chip, IBM 1988
- IEEE Restandardization: VHDL 1076-1993
- Minor change in standard 2000 and 2002
- VHDL standard IEEE 1076-2008 published in Jan 2009

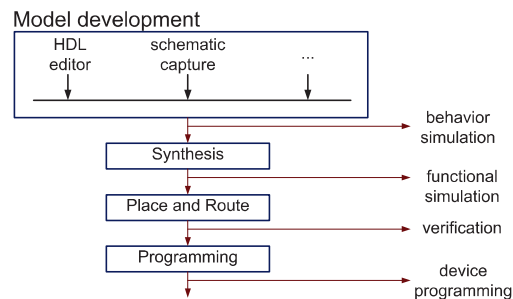
32

The role of HDL

- Formal documentation
- Input to a simulator
- Input to a synthesizer

33

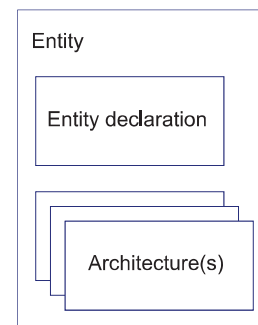
FPGA design flow



34

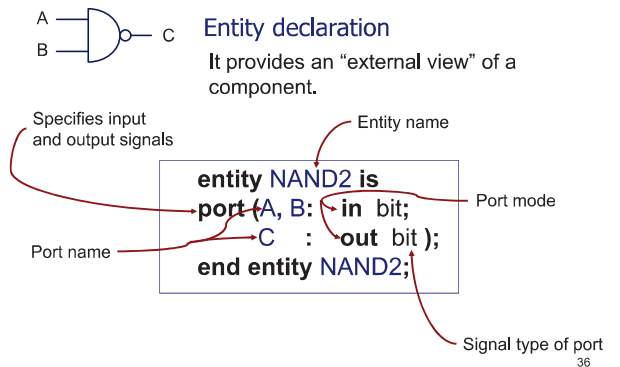
1.3: Basic VHDL Concept

- Entity**
- Architecture**
- Configuration**
- Package**
- Library**



35

NAND2 Gate



36

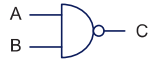
Entity Declarations

- The entity declaration defines an interface to a component
 - It names the entity, and
 - It describes the input and output ports that can be seen from the outside,
 - Mode of signals (i.e. in and out)
 - Type of signals (i.e. bit)

Possible modes for signals of entity ports

Mode	Purpose
in	Used for a signal that is an input to an entity.
out	Used for a signal that is an output from an entity. The value of the signal can not be used inside the entity. This means that in an assignment statement, the signal can appear only to the left of the <= operator.
inout	Used for a signal that is both an input to an entity and an output from the entity.
buffer	Used for a signal that is an output from an entity. The value of the signal can be used inside the entity, which means that in an assignment statement, the signal can appear both on the left and right sides of the <= operators.

NAND2 Gate



Architecture

It provides an "internal view" of a component.

Architecture name

Entity name

```
architecture DATAFLOW of NAND2 is
  signal S: bit;
begin
  S <= A and B;
  C <= not S;
end architecture DATAFLOW;
```

declaration

concurrent statements

Statements in the architecture are concurrent statements

S <= A and B;
C <= not S;

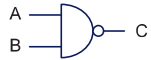
and

C <= not S;
S <= A and B;

will produce the same result



NAND2 Gate



Architecture:
Behavior style

```
architecture BEHAVIOR of NAND2 is
begin
  process (A,B) is
  begin
    if (A='1' and B='1') then
      C <= '0';
    else
      if (A='0' and B='0') or (A='0' and B='1')
        or (A='1' and B='0') then
        C <= '1';
      end if;
    end if;
  end process;
end architecture BEHAVIOR;
```

Architecture

- It defines the relationships between the inputs and outputs of a design entity.
 - It consists of a declaration section followed by a collection of *concurrent statements*.
 - It may be expressed in terms of behavior, data flow, or structure.